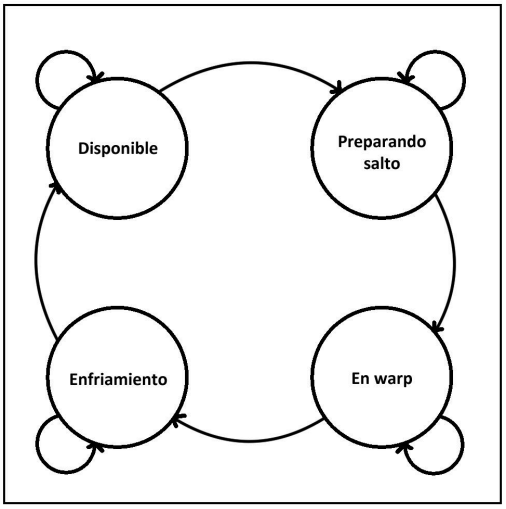
Trabajo Práctico “Misión Interestelar”

Asignatura: Programación C.

Integrantes:
Juan Pedro Gomez Ambrustolo
Valentina Sgorlo Teixeira
Matias Mercado
Joaquin Andres Ugalde

Uso de Patrones

Transiciones de Estado validas.



- Patrón State: En este proyecto lo hemos aplicado sobre el motor warp de las naves, de tal forma que los estados disponibles del motor sean: Disponible, Preparando salto, En warp y Enfriamiento. Las posibles transiciones del motor dependen de su estado actual y del *contexto* en el que se ejecuta, el cual resulta fundamental para evaluar adecuadamente las condiciones del sistema y definir el comportamiento del motor. Dicho contexto está almacenado en el atributo estadoActual que lleva la clase MotorWarp. Según aquel atributo, se llevarán a cabo las implementaciones de los métodos que llevan a transicionar entre estados; puede cambiar no solo la implementación de cada método, sino también la posibilidad de poder realizarse o no. Esto evitaría que surgieran transiciones inválidas. La idea o justificación de este diseño esta ligada a la cantidad de estados que posee el motor, y no solo eso, las transiciones entre estos no funcionan de la misma forma entre sí, hay transiciones posibles y otra inválidas. Aparte de restringir ciertas transiciones, nos permite dividir y especificar funcionalidad según cada estado.

- Patrón Factory: Dentro del paquete de las misiones, podemos ver una implementación basada en el patrón factory class. Misión es la clase padre abstracta, desde la cual se ramifican las diferentes misiones, que son creadas a partir de la clase MisionFactory. Las mismas, deben generar un informe, en este caso lo decidimos implementar como un objeto aparte creado en el momento de cerrar la misión, donde se almacenarán los recursos consumidos por la misión y el código de la misma. A posteriori se busca agregar información sobre el estado final de la misión, acciones realizadas, entre otros atributos. A su vez, este patrón también se aplicó en las clases Nave, Cargo y Origen. La principal ventaja que proporciona el aplicarlo es la de encapsular la creación de objetos, evitando exponer la lógica de instanciación al cliente (se le delega la creación a una clase Factory). Esto resulta útil en situaciones donde la clase concreta a instanciar se desconoce en tiempo de compilación.Por ejemplo, se sabe que se creará un objeto de tipo Origen, pero se desconoce qué subclase en particular será instanciada en tiempo de compilación.

- Patrón Decorator: A la hora de crear Tripulantes y asignarles un lugar de origen con su debido subsidio es la mejor opción utilizar este patrón. Al agregar “envoltorios” a los tripulantes que sean necesarios, se evita la sobrepoblación de clases en el programa. En caso de no utilizar este patrón, se tendrían que generar tantas clases como combinaciones hubiera, haciendo largo el programa innecesariamente. Este patrón permite asignar responsabilidades nuevas a una clase sin la necesidad de modificarla, respetando la O de SOLID. Esto, junto a un FactoryClass, permite facilitar en gran medida la creación de tripulantes, aislando casi completamente la lógica detrás de cada instancia.

Uso de Enumerators

En la implementación del paquete Misión podemos observar dos Enumerator que se encuentran como atributos de la misión. Los cuales funcionan como un limitador de los valores que pueden tomar los atributos correspondientes. El Estado de la misión tiene un dominio limitado, al igual que el resultado. Otra implementación válida será hardcoding con Strings, pero se vio una implementación más sólida con el uso de Enums.

Bibliografia

- https://picodotdev.github.io/blog-bitix/2021/05/el-patron-de-diseno-factory-ventajas-sobre-new-y-diferencias-con-builder/
- IA’s utilizadas: Gemini, Codex, Claude.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.